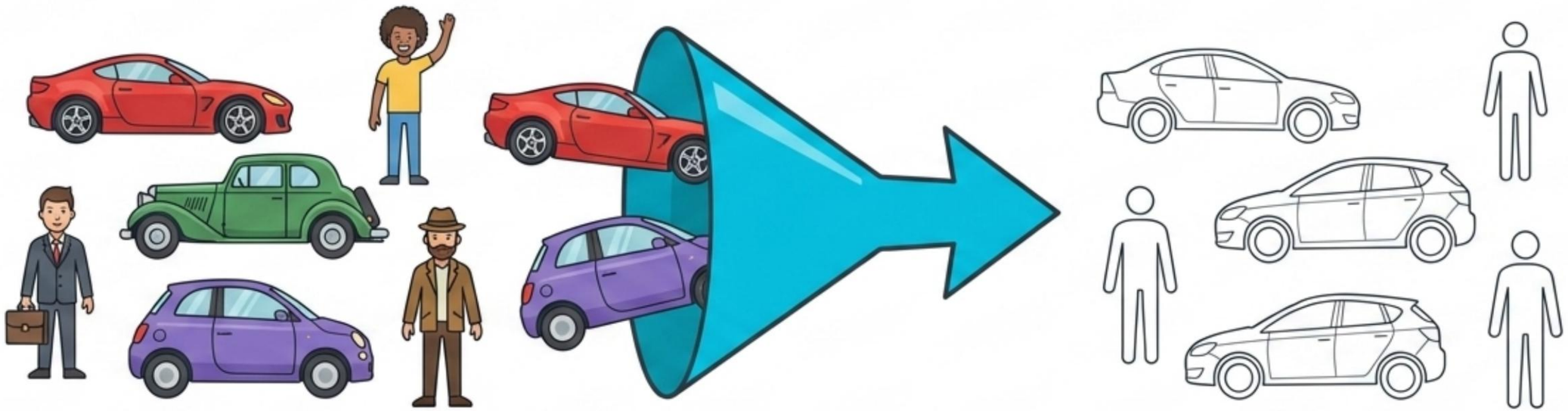


Classes Abstratas e Interfaces

Prof. Ronaldo Borges

NotebookLM

O Pilar da Abstração: Do Físico ao Molde



O Mundo Real: Complexo, cheio de detalhes específicos, propriedades únicas e ruído visual.

A Abstração (POO): A ação deliberada de focar nas características essenciais de um grupo, ignorando os detalhes irrelevantes.

Na POO, nós não programamos itens individuais. Nós criamos os moldes.

ronaldo.pb@ifpi.edu.br | (, , , p.)

NotebookLM

Classes Abstratas: Modelos Incompletos



A Regra de Ouro:

Não podem gerar objetos. É estritamente proibido usar o operador **new ClasseAbstrata()**.

O Propósito:

Servem como uma superclasse base para compartilhar um design comum e herança parcial entre classes relacionadas.

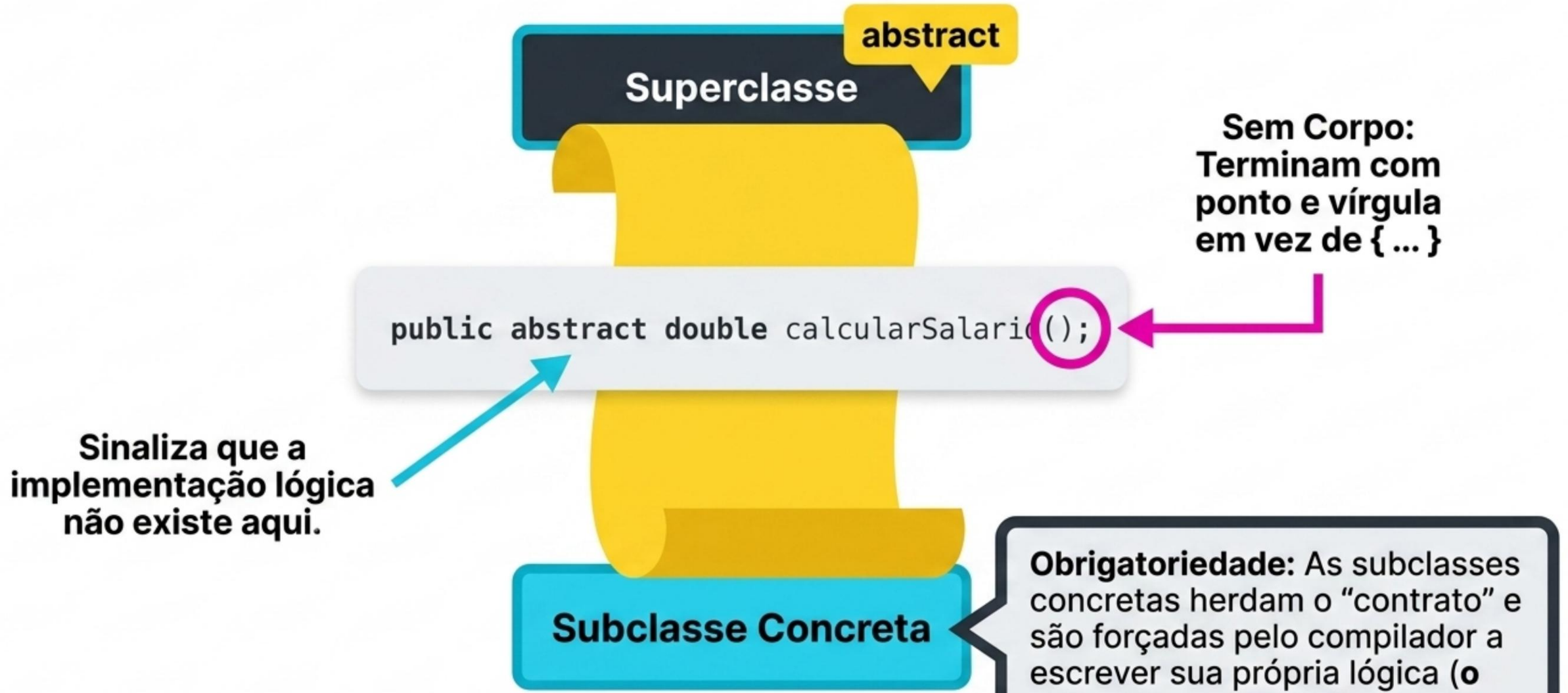
A Estrutura Híbrida:

Podem conter construtores, variáveis de estado mutável, métodos completos (concretos) e métodos vazios (abstratos).

ronaldo.pb@ifpi.edu.br | (,,, p.)

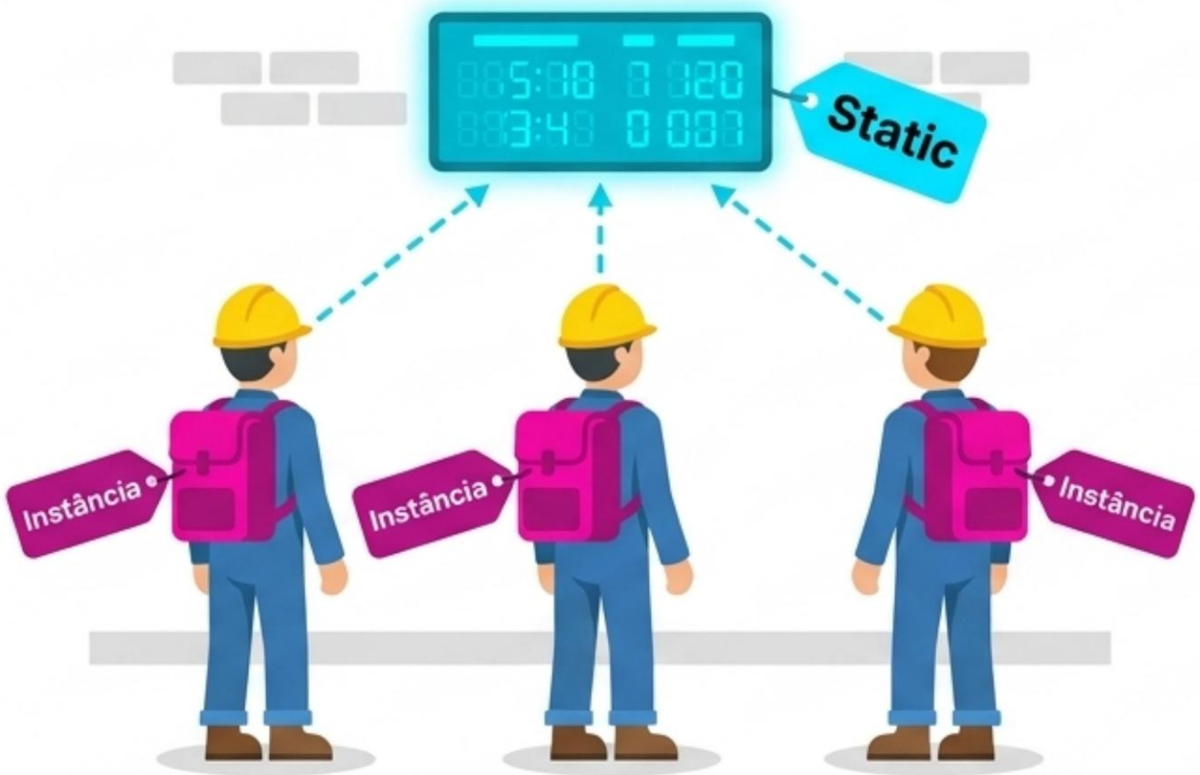
NotebookLM

Métodos Abstratos: "O Quê", mas não "O Como"



NotebookLM

Membros da Classe (static): Compartilhamento Global



Variáveis de Instância (Mochila):
Cada objeto criado na memória possui sua própria cópia exclusiva da variável. Se um altera, os outros não são afetados.

Variáveis Estáticas (Placar Digital): Escopo de classe. Todos os objetos compartilham um único local fixo na memória. O dado existe e pode ser acessado mesmo se nenhum objeto for instanciado.

Regra de Ouro: Métodos static só podem enxergar e chamar **outras variáveis** e métodos estáticos. Eles ignoram **completamente** o contexto **this** (a instância).

NotebookLM

Na Prática: Arquitetando a Base

```java
public abstract class Funcionario {
   // Compartilhado por todos
   private static int contagem = 0;
   private String nome;

   public Funcionario(String n) {
      this.nome = n;
      contagem++;
   }

   // O comportamento que varia
   public abstract double getPagamento();
}
```

Bloqueia o uso de `new Funcionario()`.

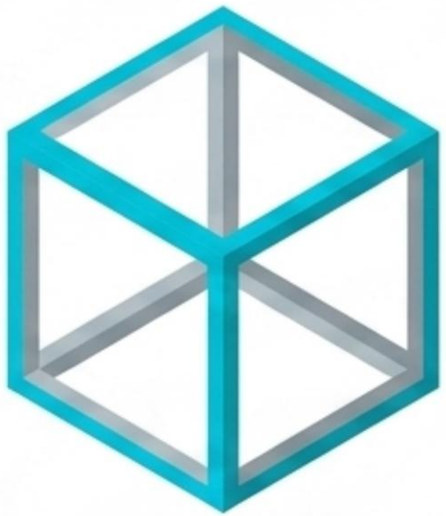
Um contador global; incrementado a cada novo funcionário real gerado na fábrica de objetos.

Delega a responsabilidade matemática do pagamento estritamente para as subclasses concretas (Ex: Horista, Comissionado).

NotebookLM

Interfaces: O Contrato Inquebrável





Uma interface define o que uma classe deve fazer, ocultando 100% de como ela fará.

Contrato Assinado: Quem implementa se responsabiliza por cumprir rigorosamente todas as regras da interface.

Isolamento: Separação total e absoluta entre a interface de comunicação externa e os mecanismos lógicos internos.

A Palavra-Chave: Em vez de **class**, usamos a instrução **interface**. Em vez de **extends**, usamos **implements**.

ronaldo.pb@ifpi.edu.br | (, , p.)

NotebookLM

A Anatomia Rígida: O Que Acontece nos Bastidores

O que você digita

```
void conectar();
int MAX = 100;
```

O Raio-X do Compilador

```
public abstract void conectar();
public static final int MAX = 100;
```

Regras de Ferro (Pré-Java 8): Uma interface não pode manter estado (variáveis de instância). Tudo nela é **puramente constante (final)** e **abstrato (abstract)**. **Não existem construtores**.

NotebookLM

Na Prática: Assinando e Cumprindo

O Contrato

```java
public interface Payable {
    // Implicitamente public abstract
    double getPaymentAmount();
}
```



A Implementação

```java
public class Fatura implements Payable {
    private double custoTotal;

    @Override
    public double getPaymentAmount() {
        return custoTotal; // O 'Como'
    }
}
```

A classe Fatura acaba de assinar um contrato garantindo ao sistema que ela possui a lógica exata de como calcular um valor de pagamento.

NotebookLM

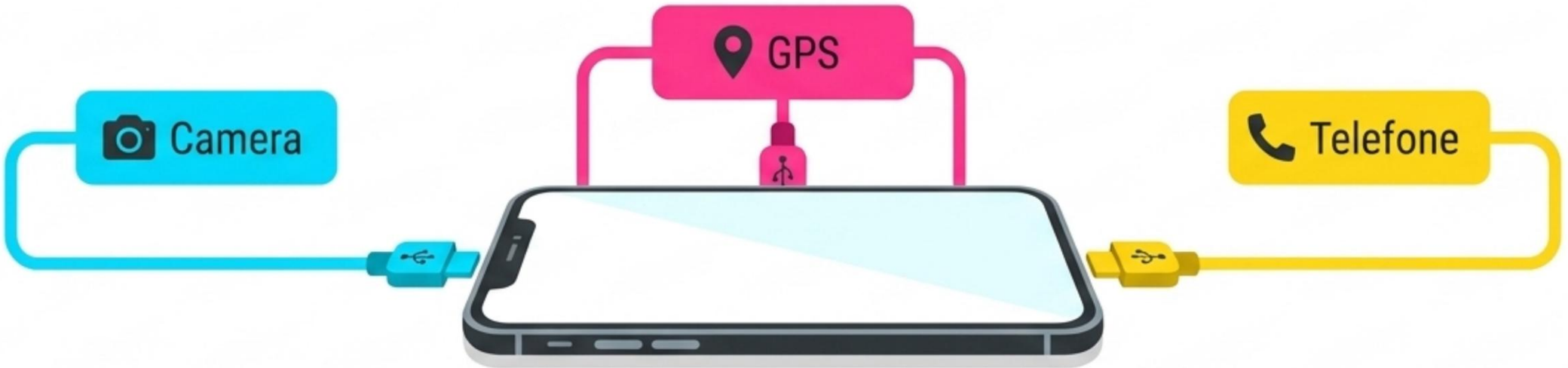
O Duelo Decisivo: Abstratas vs. Interfaces

Característica	Classe Abstrata	Interface
Instanciação	❌ Bloqueada (Sem new)	❌ Bloqueada (Sem new)
Métodos	• Podem ter corpo (concretos) e ser vazios (abstratos)	• 100% vazios / abstratos (Pré-Java 8)
Estado/Atributos	• Variáveis normais, mutáveis, protegidas ou privadas	• Apenas Constantes (public static final)
Herança (O Limite)	• Uma classe só herda UMA superclasse.	• Uma classe pode implementar MÚLTIPLAS interfaces.

NotebookLM

A Mágica da Implementação Múltipla

O Java protege o código proibindo a herança múltipla de classes lógicas.
Porém, o Java permite que uma classe assine **infinitos** contratos de comportamento!




```
public class SmartPhone extends Computador implements Camera, GPS, Telefone { ... }
```

Objetos que não possuem nenhuma relação familiar de classe podem ser agrupados e processados juntos pelo sistema, bastando que ambos implementem a mesma interface.

NotebookLM

Decisão Arquitetural: Quando usar o quê?

Use Classes Abstratas quando:

As classes têm uma relação forte de parentesco (essência estrutural).

Há necessidade de compartilhar atributos mutáveis de estado e blocos de código concretos em comum.
(Ex: Mamífero -> Cachorro).

O Desafio Arquitetural

Classes totalmente isoladas e diferentes precisam garantir a mesma funcionalidade ou o mesmo formato de comunicação para o sistema.

(Ex: Carro e Televisão são diferentes, ambos implementam ControleRemoto).

Use Interfaces quando:

Lembre-se: Classes Abstratas definem 'o que o objeto É'. Interfaces definem 'o que o objeto FAZ'.

ronaldo.pb@ifpi.edu.br | („ , p.)

NotebookLM

Referências Bibliográficas

DEITEL, P.; DEITEL, H. **Java: Como Programar.** 10. ed. São Paulo: Pearson Prentice Hall, 2016.

FURGERI, Sérgio. **Programação orientada a objetos: Conceitos e técnicas.** São Paulo: Saraiva, 2015.

SCHILDT, Herbert. **Java para iniciantes: crie, compile e execute programas Java rapidamente.** 6. ed. Porto Alegre: Bookman, 2015.

SIERRA, Kathy; BATES, Bert. **Use a cabeça! Java.** 2. ed. Rio de Janeiro: Alta Books, 2005.

TERUEL, Evandro Carlos. **Programação Orientada a Objetos com Java: sem mistérios.** 2016.

NotebookLM